

ASHESI UNIVERSITY
FACULTY OF ENGINEERING & COMPUTER SCIENCE
DEPARTMENT OF COMPUTER SCIENCE & INFORMATION SYSTEMS

CS323 DATABASE SYSTEMS

INTRO SQL ASSIGNMENT

Submitted by: Peter Mangor Buol

Student ID: 73902026

Instructor: Charles Adjetey

Date Submitted: 12 June 2026

1 . Write the SQL code to create the table structures for the entities shown in Figure 1. The structures should contain the attributes specified in the ERD. Use data types that would be appropriate for the data that will need to be stored in each attribute. Enforce primary key and foreign key constraints as indicated by the ERD.

```
mysql> use Elitevideo;
Database changed
mysql> show tables;
+-----+
| Tables_in_elitevideo |
+-----+
| detailrental          |
| membership            |
| movie                 |
| price                 |
| rental                |
| video                 |
+-----+
6 rows in set (0.01 sec)
```

```
create table Price (
price_code int primary key,
price_description varchar(20),
price_rentfee decimal(5,2),
price_dailylatefee decimal(5,2)
```

```
ERROR 1050 (42S01): Table 'price' already exists
mysql> select * from Price;
+-----+-----+-----+-----+
| price_code | price_description | price_rentfee | price_dailylatefee |
+-----+-----+-----+-----+
| 1          | Standard          | 2.50          | 1.00               |
| 2          | New Release       | 4.00          | 3.00               |
| 3          | Discount          | 2.00          | 1.00               |
| 4          | Weekly Special    | 1.50          | 0.50               |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
create table Membership (
mem_num int primary key,
mem_fname varchar(30),
```

```

mem_lname varchar(30),
mem_street varchar(50),
mem_city varchar(30),
mem_state varchar(20),
mem_zip varchar(10),
mem_balance decimal(6,2)

```

```
mysql> select * from Membership;
```

mem_num	mem_fname	mem_lname	mem_street	mem_city	mem_state	mem_zip	mem_balance
102	Tami	Dawson	2632 Takli Circle	Norene	TN	37136	11.00
103	Curt	Knight	4025 Cornell Court	Flatgap	KY	41219	6.00
104	Jamal	Melendez	788 East 145th Avenue	Quebeck	TN	38579	0.00
105	Iva	Mcclain	6045 Musket Ball Circle	Summit	KY	42783	15.00
106	Miranda	Parks	4469 Maxwell Place	Germantown	TN	38183	0.00
107	Rosario	Elliott	7578 Danner Avenue	Columbia	TN	38402	5.00
108	Mattie	Guy	4390 Evergreen Street	Lily	KY	40740	0.00
109	Clint	Ochoa	1711 Elm Street	Greeneville	TN	37745	10.00
110	Lewis	Rosales	4524 Southwind Circle	Counce	TN	38326	0.00
111	Stacy	Mann	2789 East Cook Avenue	Murfreesboro	TN	37132	8.00
112	Luis	Trujillo	7267 Melvin Avenue	Heiskell	TN	37754	3.00
113	Minnie	Gonzales	6430 Vasili Drive	Williston	TN	38076	0.00

```

create table detailrental (
rent_num int,
vid_num int,
detail_fee decimal(5,2),
detail_duedate date,
detail_returndate date,
detail_dailylatefee decimal(5,2),
primary key (rent_num, vid_num),
foreign key (rent_num) references rental(rent_num),
foreign key (vid_num) references video(vid_num)
);

```

```
mysql> select * from detailrental;
```

rent_num	vid_num	detail_fee	detail_duedate	detail_dailylatefee	detail_returndate
1001	34342	2.00	2016-03-04	NULL	2016-03-02
1001	34366	3.50	2016-03-04	3.00	2016-03-02
1001	61353	2.00	2016-03-04	1.00	2016-03-03
1002	59237	3.50	2016-03-04	3.00	2016-03-04
1003	54325	3.50	2016-03-04	3.00	2016-03-09
1003	61369	2.00	2016-03-06	1.00	2016-03-09
1003	61388	0.00	2016-03-06	1.00	2016-03-09
1004	34341	2.00	2016-03-07	1.00	2016-03-07
1004	34367	3.50	2016-03-05	3.00	2016-03-07
1004	44392	3.50	2016-03-05	3.00	2016-03-07
1005	34342	2.00	2016-03-07	1.00	2016-03-05
1005	44397	3.50	2016-03-05	3.00	2016-03-05
1006	34366	3.50	2016-03-05	3.00	2016-03-04
1006	61367	2.00	2016-03-07	1.00	NULL
1007	34368	3.50	2016-03-05	3.00	NULL
1008	34369	3.50	2016-03-05	3.00	2016-03-05
1009	54324	3.50	2016-03-05	3.00	NULL

```
17 rows in set (0.00 sec)
```

```
create table video (
vid_num int primary key,
vid_indate date,
movie_num int,
foreign key (movie_num) references movie(movie_num) );
```

```
mysql> select * from Video;
```

vid_num	vid_indate	movie_num
34341	2014-01-22	1235
34342	2014-01-22	1235
34366	2016-03-02	1236
34367	2016-03-02	1236
34368	2016-03-02	1236
34369	2016-03-02	1236
44392	2015-10-21	1237
44397	2015-10-21	1237
54321	2015-06-18	1234
54324	2015-06-18	1234
54325	2015-06-18	1234
59237	2016-02-14	1237
61353	2013-01-28	1245
61354	2013-01-28	1245
61367	2015-07-30	1246
61369	2015-07-30	1246
61388	2014-01-25	1239

```
17 rows in set (0.00 sec)
```

- Write the SQL command to change the movie year for movie number 1245 to 2010.
Show the result before and after the change.

```
mysql> select * from Movie;
+-----+-----+-----+-----+-----+-----+
| movie_num | movie_title | movie_year | movie_cost | movie_genre | price_code |
+-----+-----+-----+-----+-----+-----+
| 1234 | The Cesar Family Christmas | 2014 | 39.95 | FAMILY | 2 |
| 1235 | Smokey Mountain Wildlife | 2011 | 59.95 | ACTION | 1 |
| 1236 | Richard Goodhope | 2015 | 59.95 | DRAMA | 2 |
| 1237 | Beatnik Fever | 2014 | 29.95 | COMEDY | 2 |
| 1238 | Constant Companion | 2015 | 89.95 | DRAMA | NULL |
| 1239 | Where Hope Dies | 2005 | 25.49 | DRAMA | 3 |
| 1245 | Time to Burn | 2012 | 45.49 | ACTION | 1 |
| 1246 | What He Doesn't Know | 2013 | 58.29 | COMEDY | 1 |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> select * from Movie
-> where movie_num = 1245;
+-----+-----+-----+-----+-----+-----+
| movie_num | movie_title | movie_year | movie_cost | movie_genre | price_code |
+-----+-----+-----+-----+-----+-----+
| 1245 | Time to Burn | 2012 | 45.49 | ACTION | 1 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Updated one.

```
mysql> update Movie
-> set movie_year = 2010
-> where movie_num = 1245;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select * from Movie
-> where movie_num = 1245;
+-----+-----+-----+-----+-----+-----+
| movie_num | movie_title | movie_year | movie_cost | movie_genre | price_code |
+-----+-----+-----+-----+-----+-----+
| 1245 | Time to Burn | 2010 | 45.49 | ACTION | 1 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

4. Write the SQL command to change the price code for all Action movies to price code 3.
3. Show the result before and after the change.

```
mysql> select * from Movie
-> where movie_genre = 'ACTION';
```

movie_num	movie_title	movie_year	movie_cost	movie_genre	price_code
1235	Smokey Mountain Wildlife	2011	59.95	ACTION	1
1245	Time to Burn	2010	45.49	ACTION	1

```
2 rows in set (0.00 sec)
```

```
mysql> update Movie
-> set price_code = 3
-> where movie_genre = 'ACTION';
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2 Changed: 2 Warnings: 0
```

```
mysql> select * from Movie
-> where movie_genre = 'ACTION';
```

movie_num	movie_title	movie_year	movie_cost	movie_genre	price_code
1235	Smokey Mountain Wildlife	2011	59.95	ACTION	3
1245	Time to Burn	2010	45.49	ACTION	3

```
2 rows in set (0.00 sec)
```

5. Write a single SQL command to increase all price rental fee values in the PRICE table by \$0.50. Show the result before and after the change.

```
mysql> select * from Price;
```

price_code	price_description	price_rentfee	price_dailylatefee
1	Standard	2.00	1.00
2	New Release	3.50	3.00
3	Discount	1.50	1.00
4	Weekly Special	1.00	0.50

```
4 rows in set (0.00 sec)
```

```
mysql> update Price
-> set price_rentfee = price_rentfee + $0.50;
ERROR 1054 (42S22): Unknown column '$0.50' in 'field list'
mysql> update Price
-> set price_rentfee = price_rentfee + 0.50;
Query OK, 4 rows affected (0.01 sec)
Rows matched: 4 Changed: 4 Warnings: 0
```

```
mysql> select * from price;
```

price_code	price_description	price_rentfee	price_dailylatefee
1	Standard	2.50	1.00
2	New Release	4.00	3.00
3	Discount	2.00	1.00
4	Weekly Special	1.50	0.50

```
4 rows in set (0.00 sec)
```

6. Write a query to display the movie title, movie year, and movie genre for all movies (result shown in Figure 2).

```
mysql> select movie_title, movie_year, movie_genre
-> from Movie;
```

movie_title	movie_year	movie_genre
The Cesar Family Christmas	2014	FAMILY
Smokey Mountain Wildlife	2011	ACTION
Richard Goodhope	2015	DRAMA
Beatnik Fever	2014	COMEDY
Constant Companion	2015	DRAMA
Where Hope Dies	2005	DRAMA
Time to Burn	2010	ACTION
What He Doesn't Know	2013	COMEDY

```
8 rows in set (0.00 sec)
```


7. Write a query to display the movie year, movie title, and movie cost sorted by movie year in descending order

```
8 rows in set (0.00 sec)

mysql> select movie_year, movie_title, movie_cost
      -> from Movie
      -> order by movie_year desc;
+-----+-----+-----+
| movie_year | movie_title          | movie_cost |
+-----+-----+-----+
| 2015      | Richard Goodhope    | 59.95      |
| 2015      | Constant Companion   | 89.95      |
| 2014      | The Cesar Family Christmas | 39.95      |
| 2014      | Beatnik Fever       | 29.95      |
| 2013      | What He Doesn't Know | 58.29      |
| 2011      | Smokey Mountain Wildlife | 59.95      |
| 2010      | Time to Burn        | 45.49      |
| 2005      | Where Hope Dies     | 25.49      |
+-----+-----+-----+
8 rows in set (0.00 sec)
```

8. Write a query to display the movie title, movie year, and movie genre for all movies sorted by movie genre in ascending order, then sorted by movie year in descending order within genre

```
mysql> select movie_title, movie_year, movie_genre
-> from Movie
-> order by movie_genre, movie_year desc;
```

movie_title	movie_year	movie_genre
Smokey Mountain Wildlife	2011	ACTION
Time to Burn	2010	ACTION
Beatnik Fever	2014	COMEDY
What He Doesn't Know	2013	COMEDY
Richard Goodhope	2015	DRAMA
Constant Companion	2015	DRAMA
Where Hope Dies	2005	DRAMA
The Cesar Family Christmas	2014	FAMILY

```
8 rows in set (0.00 sec)
```

9. Write a query to display the movie title, movie year, and movie genre for all movies sorted by movie genre in ascending order, then sorted by movie year in descending order within genre

```
mysql> select from movie_num, movie_title, price_code
-> from Movie where movie_title like 'R%';
ERROR 1064 (42000): You have an error in your SQL syntax; check the
right syntax to use near 'from movie_num, movie_title, price_code
from Movie where movie_title like 'R%' at line 1
mysql> select movie_num, movie_title, price_code
-> from Movie where movie_title like 'R%';
```

movie_num	movie_title	price_code
1236	Richard Goodhope	2

```
1 row in set (0.00 sec)

mysql> |
```

10. Write a query to display the movie title, movie year, and movie cost for all movies that contain the word “hope” anywhere in the title. Sort the results in ascending order by title

```
mysql> select movie_title, movie_year, movie_cost
-> from Movie
-> where movie_title like 'HOPE%'
-> order by movie_title asc;
Empty set (0.00 sec)
```

```
mysql> select movie_title, movie_year, movie_cost
-> from Movie
-> where movie_title like '%HOPE%'
-> order by movie_title asc;
```

movie_title	movie_year	movie_cost
Richard Goodhope	2015	59.95
Where Hope Dies	2005	25.49

2 rows in set (0.00 sec)

11. Write a query to display the movie title, movie year, and movie genre for all action movies

```
mysql> select movie_title, movie_year, movie_genre
-> from Movie
-> where movie_genre = 'ACTION';
```

movie_title	movie_year	movie_genre
Smokey Mountain Wildlife	2011	ACTION
Time to Burn	2010	ACTION

2 rows in set (0.03 sec)

12. Write a query to display the movie number, movie title, and movie cost for all movies with a cost greater than \$40

```
mysql> select movie_num, movie_title, movie_cost
-> where movie_cost > 40;
ERROR 1054 (42S22): Unknown column 'movie_num' in 'field list'
mysql> select movie_num, movie_title, movie_cost
-> from Movie
-> where movie_cost > 40;
```

movie_num	movie_title	movie_cost
1235	Smokey Mountain Wildlife	59.95
1236	Richard Goodhope	59.95
1238	Constant Companion	89.95
1245	Time to Burn	45.49
1246	What He Doesn't Know	58.29

```
5 rows in set (0.00 sec)
```

13. Write a query to display the movie number, movie title, movie cost, and movie genre for all movies that are either action or comedy movies and that have a cost that is less than \$50. Sort the results in ascending order by genre.

```
mysql> select movie_num, movie_title, movie_cost, movie_genre
-> from Movie
-> where (movie_genre = 'ACTION' or movie_genre = 'COMEDY')
-> and movie_cost < 50
-> order by movie_genre asc;
```

movie_num	movie_title	movie_cost	movie_genre
1245	Time to Burn	45.49	ACTION
1237	Beatnik Fever	29.95	COMEDY

```
2 rows in set (0.00 sec)
```

14. Write a query to display the membership number, name, street, state, and balance for all members in Tennessee (TN), with a balance less than \$5, and whose street name ends in "Avenue."

```
mysql> select mem_num, mem_Fname, mem_Lname, mem_street, mem_state, mem_balance
-> from Membership
-> where mem_state = 'TN' and mem_balance < 5 and mem_street like '%Avenue';
```

mem_num	mem_Fname	mem_Lname	mem_street	mem_state	mem_balance
104	Jamal	Melendez	788 East 145th Avenue	TN	0.00
112	Luis	Trujillo	7267 Melvin Avenue	TN	3.00

2 rows in set (0.00 sec)

15. Write a query to display the movie genre and the number of movies in each genre (result shown in Figure 11).

```
mysql> select movie_genre,
-> count(*) as number_of_movies
-> from Movie group by movie_genre;
```

movie_genre	number_of_movies
FAMILY	1
ACTION	2
DRAMA	3
COMEDY	2

4 rows in set (0.01 sec)

16. Write a query to display the average cost of all of the movies (result shown in Figure 12).

```
mysql> select avg(movie_cost) as average_movie_cost
-> from Movie;
```

average_movie_cost
51.127500

1 row in set (0.00 sec)

17. Write a query to display the movie genre and average cost of movies in each genre (result shown in Figure P7.13).

```
mysql> select movie_genre
-> avg(movie_cost) as average_cost
-> from Movie
-> group by movie_genre;
ERROR 1064 (42000): You have an error in your SQL syntax; c
ight syntax to use near '(movie_cost) as average_cost
from Movie
group by movie_genre' at line 2
mysql> select movie_genre,
-> avg(movie_cost) as average_cost
-> from Movie
-> group by movie_genre;
```

movie_genre	average_cost
FAMILY	39.950000
ACTION	52.720000
DRAMA	58.463333
COMEDY	44.120000

4 rows in set (0.00 sec)

18. Write a query to display the movie title, movie genre, price description, and price rental fee for all movies with a price code (result shown in Figure 14).

```
mysql> select movie_title, movie_genre, price_description, price_rentfee
-> from Movie
-> join price
-> on Movie.price_code = Price.price_code;
```

movie_title	movie_genre	price_description	price_rentfee
What He Doesn't Know	COMEDY	Standard	2.50
The Cesar Family Christmas	FAMILY	New Release	4.00
Richard Goodhope	DRAMA	New Release	4.00
Beatnik Fever	COMEDY	New Release	4.00
Smokey Mountain Wildlife	ACTION	Discount	2.00
Where Hope Dies	DRAMA	Discount	2.00
Time to Burn	ACTION	Discount	2.00

7 rows in set (0.02 sec)

19. . Write a query to display the movie genre and average price rental fee for movies in each genre that have a price (result shown in Figure 15).

```
mysql> select movie_genre,
-> avg(price_rentfee) as average_rental_fee
-> from Movie
-> join Price
-> on Movie.price_code = Price.price_code
-> group by movie_genre;
```

movie_genre	average_rental_fee
COMEDY	3.250000
FAMILY	4.000000
DRAMA	3.000000
ACTION	2.000000

4 rows in set (0.00 sec)

20. Write a query to display the movie title and the movie cost divided by the price rental fee for each movie that has a price to determine the number of rentals it will take to break even on the purchase of the movie (result shown in Figure 16).

```
mysql> select movie_title, movie_cost, price_rentfee
-> movie_cost / price_rentfee as break_even_rentals
-> from Movie
-> join Price
-> on Movie.price_code = Price.price_code;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL
ight syntax to use near '/' price_rentfee as break_even_rentals
from Movie
join Price
on Movie.price_code ' at line 2
mysql> select movie_title, movie_cost, price_rentfee, movie_cost /price_rentfee as break_even_rentals
-> from Movie
-> join price on Movie.price_code = price.price_code;
```

movie_title	movie_cost	price_rentfee	break_even_rentals
What He Doesn't Know	58.29	2.50	23.316000
The Cesar Family Christmas	39.95	4.00	9.987500
Richard Goodhope	59.95	4.00	14.987500
Beatnik Fever	29.95	4.00	7.487500
Smokey Mountain Wildlife	59.95	2.00	29.975000
Where Hope Dies	25.49	2.00	12.745000
Time to Burn	45.49	2.00	22.745000

7 rows in set (0.00 sec)

21. Write a query to display the movie title and movie year for all movies that have a price code

```
mysql> select movie_title, movie_year
-> from Movie
-> where price_code is NOT NULL;
```

movie_title	movie_year
What He Doesn't Know	2013
The Cesar Family Christmas	2014
Richard Goodhope	2015
Beatnik Fever	2014
Smokey Mountain Wildlife	2011
Where Hope Dies	2005
Time to Burn	2010

7 rows in set (0.00 sec)

22. Write a query to display the movie title, movie genre, and movie cost for all movies that have a cost between \$44.99 and \$49.99.


```
mysql> select movie_title, movie_genre, movie_cost
-> from Movie
-> where movie_cost between 44.99 and 49.99;
+-----+-----+-----+
| movie_title | movie_genre | movie_cost |
+-----+-----+-----+
| Time to Burn | ACTION      | 45.49      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

23. Write a query to display the movie title, price description, and price rental fee for all movies that are in the genres Family, Comedy, or Drama (result shown in Figure 19).

```
mysql> select movie_title, price_description, price_rentfee,
-> from Movie
-> join Movie.price_code = Price.price_code
-> where movie_genre in ('FAMILY', 'COMEDY', 'DRAMA');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version
right syntax to use near 'from Movie
join Movie.price_code = Price.price_code
where movie_genre in ('FAMIL' at line 2
mysql> SELECT movie_title,
-> price_description,
-> price_rentfee
-> FROM MOVIE
-> JOIN PRICE
-> ON MOVIE.price_code = PRICE.price_code
-> WHERE movie_genre IN ('FAMILY', 'COMEDY', 'DRAMA');
+-----+-----+-----+
| movie_title | price_description | price_rentfee |
+-----+-----+-----+
| The Cesar Family Christmas | New Release      | 4.00          |
| Richard Goodhope           | New Release      | 4.00          |
| Beatnik Fever              | New Release      | 4.00          |
| Where Hope Dies            | Discount         | 2.00          |
| What He Doesn't Know       | Standard         | 2.50          |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

24. Write a query to display the membership number, first name, last name, and balance of the memberships that have a rental.

```
mysql> select distinct membership.mem_num, mem_fname, mem_lname, mem_balance
-> from membership
-> join rental
-> on membership.mem_num = rental.mem_num;
+-----+-----+-----+-----+
| mem_num | mem_fname | mem_lname | mem_balance |
+-----+-----+-----+-----+
| 102 | Tami | Dawson | 11.00 |
| 103 | Curt | Knight | 6.00 |
| 104 | Jamal | Melendez | 0.00 |
| 105 | Iva | McClain | 15.00 |
| 107 | Rosario | Elliott | 5.00 |
| 110 | Lewis | Rosales | 0.00 |
| 111 | Stacy | Mann | 8.00 |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

25. Write a query to display the minimum balance, maximum balance, and average balance for memberships that have a rental (result shown in Figure 21).

```
mysql> select min(mem_balance) as min_balance, max
-> select min(mem_balance) as min_balance, max
-> \c
mysql> select min(mem_balance) as min_balance, max(mem_balance) as max_balance, avg(mem_balance) as averg_balance
-> from Membership
-> join Rental
-> on Membership.mem_num = Rental.mem_num;
+-----+-----+-----+
| min_balance | max_balance | averg_balance |
+-----+-----+-----+
| 0.00 | 15.00 | 7.555556 |
+-----+-----+-----+
1 row in set (0.01 sec)

mysql>
```

26. Write a query to display the rental number, rental date, video number, movie title, due date, and return date for all videos that were returned after the due date. Sort the results by rental number and movie title

```
mysql> select rental.rent_num, rental.rent_date, movie.movie_title, detailrental.detail_fee
-> from rental
-> join detailrental on rental.rent_num = detailrental.rent_num
-> join video on detailrental.vid_num = video.vid_num
-> join movie on video.movie_num = movie.movie_num
-> where detailrental.detail_returndate <= detailrental.detail_duedate;
```

rent_num	rent_date	movie_title	detail_fee
1001	2016-03-01	Smokey Mountain Wildlife	2.00
1001	2016-03-01	Richard Goodhope	3.50
1001	2016-03-01	Time to Burn	2.00
1002	2016-03-01	Beatnik Fever	3.50
1004	2016-03-02	Smokey Mountain Wildlife	2.00
1005	2016-03-02	Smokey Mountain Wildlife	2.00
1005	2016-03-02	Beatnik Fever	3.50
1006	2016-03-02	Richard Goodhope	3.50
1008	2016-03-03	Richard Goodhope	3.50

9 rows in set (0.00 sec)

27. Write a query to display the rental number, rental date, movie title, and detail fee for each movie that was returned on or before the due date (result shown in Figure 23).

```
mysql> select rental.rent_num,
-> rental.rent_date,
-> movie.movie_title,
-> detailrental.detail_fee
-> from rental
-> join detailrental
-> on rental.rent_num = detailrental.rent_num
-> join video
-> on detailrental.vid_num = video.vid_num
-> join movie
-> on video.movie_num = movie.movie_num
-> where detailrental.detail_returndate <= detailrental.detail_duedate;
```

rent_num	rent_date	movie_title	detail_fee
1001	2016-03-01	Smokey Mountain Wildlife	2.00
1001	2016-03-01	Richard Goodhope	3.50
1001	2016-03-01	Time to Burn	2.00
1002	2016-03-01	Beatnik Fever	3.50
1004	2016-03-02	Smokey Mountain Wildlife	2.00
1005	2016-03-02	Smokey Mountain Wildlife	2.00
1005	2016-03-02	Beatnik Fever	3.50
1006	2016-03-02	Richard Goodhope	3.50
1008	2016-03-03	Richard Goodhope	3.50

9 rows in set (0.00 sec)

28. Write a query to display the movie number, movie genre, average movie cost of movies in that genre, movie cost of that individual movie, and the percentage difference between

the average movie cost and the individual movie cost (result shown in Figure 24). Note: the percentage difference is calculated as the cost of the individual movie minus the average cost of movies in that genre, divided by the average cost of movies in that genre multiplied by 100. For example, if the average cost of movies in the “Family” genre is \$25, if a given Family movie cost \$26, then the calculation would be $((26 - 25) / 25 * 100)$, which would work out to be 4.00%. This indicates that this movie costs 4% more than the average Family movie.

```
mysql> select m.movie_num,
->      m.movie_genre,
->      g.avg_cost,
->      m.movie_cost,
->      ((m.movie_cost - g.avg_cost) / g.avg_cost) * 100 as percent_difference
-> from movie m
-> join
-> (
->   select movie_genre,
->         avg(movie_cost) as avg_cost
->   from movie
->   group by movie_genre
-> ) g
-> on m.movie_genre = g.movie_genre;
```

movie_num	movie_genre	avg_cost	movie_cost	percent_difference
1234	FAMILY	39.950000	39.95	0.0000000000
1235	ACTION	52.720000	59.95	13.7139605463
1236	DRAMA	58.463333	59.95	2.5429049692
1237	COMEDY	44.120000	29.95	-32.1169537625
1238	DRAMA	58.463333	89.95	53.8571192990
1239	DRAMA	58.463333	25.49	-56.4000225577
1245	ACTION	52.720000	45.49	-13.7139605463
1246	COMEDY	44.120000	58.29	32.1169537625

